IE 678: Deep Learning

Assignment No. 3 (FSS 2026)

Prof. Dr.-Ing. Margret Keuper

University of Mannheim
Chair of Machine Learning

Task	Maximum	Points
1	4	.
2	4	.
3	4	.
4	15	.
5	12	.
Total	39	.

Rules for Submission

a) Hand-in your solution via ILIAS until the date specified there. This is a hard deadline.

b) Pair submission:

   (i) We strongly recommend to work on this assignment in pairs—i.e, with one (and only one) additional student—and then hand in a pair submission.

   (ii) Both students must submit exactly the same contents.

c) Code:

   (i) You must submit all relevant code files (Python scripts etc.) that we require to run your code and to replicate your results.

   (ii) In particular: You must submit all Python scripts that we provided to you.

   (iii) You must include a PDF of every Jupyter notebook in which the most relevant cells are executed (i.e., make sure to disable excessively verbose cells that inflate the page count).

   (iv) It suffices to fill out the "holes" that are marked in the code fragments.

d) Non-code parts of assignment:

   (i) You must provide your answers by filling out the provided solution LaTeX file.

   (ii) You must submit your solution sheet (where you provided your answers) as a single PDF file (do not submit LaTeX source code). All answers must be provided *inside* the boxes (i.e., no resizing of boxes, no overflow, no additional pages, no modification of font (size) etc.) specified therein.

   (iii) You may add any LaTeX packages that you absolutely need.

   (iv) You must fill out the AI declaration at the end of the solution file following the provided instructions.

   (v) The solution PDF and all notebooks must clearly report the name and matriculation number of both students right at the beginning.

   (vi) Name your solution PDF as follows:
   `a0[assignment number]-[ilias name 1]-[ilias name 2].pdf`
   For instance: `a01-gmayer-tswift.pdf`

We reserve the right to refuse to grade any assignment that violates any of these rules.

Task 1: Datasets — 4 Points

PyTorch provides an abstract `torch.utils.data.Dataset` class to represent datasets, each consisting of a set of keys and corresponding data samples. Although PyTorch also provides a number of subclasses and default implementations for common cases (recall the previous assignment's `TensorDataset`), you are going to write your own dataset from scratch in this task.

We provide the stub class `ReviewsDataset` to get started. Here, each key corresponds to the item number (line number in raw file, 0-based) and data samples to the corresponding (movie review, label)-pair. Your task is to complete this class by ultimately implementing both the `__len__` and the `__getitem__` methods (see `https://pytorch.org/docs/stable/data.html#torch.utils.data.Dataset`).

a) Preprocess the data. To do so, **complete** the `_preprocess_reviews` method (to tokenize the reviews and remove punctuation) and the `_preprocess_labels` (to convert labels to binary values). (2 points)

b) **Implement** the `__len__` and `__getitem__` methods and test your code. (1 point)

c) Observe that the provided code builds a vocabulary `self.vocab` (`https://pytorch.org/text/stable/vocab.html`), i.e., a bijective mapping from textual *tokens* to numeric *token IDs*. **Change** your code so that `__getitem__` returns a list of token ids instead of a list of tokens when `use_vocab=True`. Test your code. (1 point)

Task 2: Data Loaders — 4 Points

To process data during training, PyTorch makes use of a `torch.utils.data.DataLoader`.

a) Familiarize yourself with the documentation and run the provided example dataloader. Now, modify the dataloader by increasing the batch size. You will notice that the dataloader now fails.

b) PyTorch dataloaders accept a so-called `collate_fn` to perform modifications to each batch in the data loader (e.g., data augmentation). Here we will use the collate function to pad or trim all reviews to the same number of tokens:

   (i) **Add** zero padding (token `<pad>`, token id 0) for all reviews that are shorter than `MAX_SEQ_LEN`.

   (ii) **Crop** all reviews longer then `MAX_SEQ_LEN`.[1] (2 points)

   Complete the `reviews_collate_fn` provided to you and construct data loaders for training, validation, and testing.

c) **Explain briefly** the different purposes of (i) a dataset and (ii) a data loader in PyTorch. (2 points)

[1] Alternatively and more realistically, one may pad all reviews to the length of the longest review in the current batch (referred to as *dynamic batching*). However, this may increase the computational complexity and might require a GPU for training sufficiently fast, so we don't do this here.

Task 3: Recurrent Neural Networks 4 Points

You have now prepared that data needed for training. In this task, proceed to implement an RNN encoder (which you will train in the next task). You are going to use PyTorch's pre-canned layer implementations (see `torch.nn`). In particular, you will use (in this order):

- Embedding layer (`torch.nn.Embedding`) to embed input tokens.
- LSTM encoder (`torch.nn.LSTM`) to produce a thought vector (= last-layer hidden state of last input token).[2]
- Linear layer with one hidden unit (`torch.nn.Linear`) to produce logit scores from the thought vector.
- Logistic function (`torch.nn.Sigmoid`) to produce the model's probability of a positive sentiment.

Consult the PyTorch documentation on how these modules are to be used. Pay close attention to the LSTM module and its "inputs" section which describes the required shape of the input sequence and the initial hidden states. We provide a stub implementation called `SimpleLSTM`.

a) **Complete** the `init_hidden` method. (1 point)

b) **Complete** the constructor. To do so, **instantiate** all of the individual components given above in `__init__`. (1 point)

c) **Implement** the `forward` pass. The method obtains a batch from the data loader as input, and should return a tuple consisting of:

   (i) A tensor of predicted probabilities for the positive class (shape: `batch_size` $\times$ 1). This is the model output.

   (ii) A tensor of the thought vectors (shape: `batch_size` $\times$ `hidden_dim`). We will use this for visualization later on. (2 points)

[2] We will use dropout to regularize the LSTM encoder. Simply set the `dropout` argument of the encoder accordingly (see provided code).

Task 4: PyTorch Lightning — 15 Points

In this task, you will familiarize yourself with PyTorch Lightning, a library which provides higher-level features on top of PyTorch. In particular, Lightning bundles information about training and evaluation within each module and provides functionality such as training loops, early stopping, gradient clipping, learning rate scheduling, and so on.

Some of the following subtasks take only one line of implementation. So do not be surprised when your solutions are short and simple: that is why we want to use Lightning in the first place.

a) Watch this (two minute) video (`https://lightning.ai/docs/pytorch/stable/_static/fetched-s3-assets/pl_readme_gif_2_0.mp4`) video that gives you a brief overview of the central concepts of `LightningModules`.

b) Consider the class `LitSimpleLSTM`. First, **complete** the implementation of `configure_optimizers` by (i) defining and (ii) returning an optimizer. Use `torch.optim.Adam` and the learning rate stored as `self.lr`. (1 point)

   Hint: Use the property `requires_grad` of the `model.parameters()` to determine which parameters the optimizer needs to update (relevant later for the next task).

c) **Complete** the implementation of the `forward` method. (1 point)

d) **Complete** the implementation of the methods `training_step` and `_eval`. These methods contain only the logic for a processing a single training of evaluation batch, respectively. This means you need to:

   (i) Process the batch using the model defined in the constructor.

   (ii) Compute the loss using `binary_cross_entropy` in both methods.

   (iii) Using the inherited method `self.log`, log the loss and the accuracy per epoch in both `training_step` and `_eval`.

   (iv) Additionally return the loss in `training_step`. (5 points)

e) Watch this (one minute) video (`https://lightning.ai/docs/pytorch/stable/_static/fetched-s3-assets/pt_dm_vid.mp4`) that explains the central concepts of a `LightningDataModule`.

f) **Complete** the implementation of the `ReviewsDataModule`, which wraps your reviews dataset and its associated logic—e.g., train, validation and test splits, batching, and data loaders—inside a single class which inherits from `LightningDataModule`. (4 points)

We will train the model and monitor this process through TensorBoard, which provides logging facilities and a web interface to monitor and analyze neural network training.

g) **Initialize** a Lightning `Trainer` with the following settings:

   - Clip gradients at $v = 3$.
   - Train for (at most) 10 epochs.
   - Run validation every epoch.
   - Log the training progress to the directory `logs` using a `TensorBoardLogger`.

   First, perform a sanity check of your implementation by setting `fast_dev_run` to `True` in the Lightning `Trainer`. If you see no errors, disable this and train your model for a few epochs. (2 points)

h) Monitor your training progress in TensorBoard. To access TensorBoard in a local environment, use `tensorboard -logdir=logs` in a shell in your code's directory, and open `localhost:6006` in your browser. Alternatively, add a cell with:

```
%load_ext tensorboard
%tensorboard -logdir logs
```

   This opens TensorBoard directly inside your notebook.

i) **Discuss** (i) two advantages and (ii) two disadvantages that you have encountered in this task with Lightning in comparison to the training loop implementation of the previous assignment. (2 points)

j) **Optional.** Check out the documentation of the Lightning `Trainer` and tweak your training setup by trying out different settings.

Task 5: Pretrained Embeddings & Visualization 12 Points

You are provided with GloVe word embeddings in the file `data/word-embeddings.txt`.[3]

a) **Describe** and explain the content of the file `data/word-embeddings.txt`. (2 points)

b) Load the pretrained embeddings for the subset of the words used in this assignment into a plain embedding layer (code provided). Then, **explore** the embeddings using (i) t-SNE and (ii) another method of your choice (e.g., cosine similarities). **Discuss** your findings using suitable visualizations. (4 points)

c) Train the provided `LitSimpleLSTM` model (with the provided hyperparameter settings) for 10 epochs. Then use t-SNE to **visualize:**

   (i) The embeddings of the thought vectors of the training dataset.

   (ii) The embeddings of the thought vectors of the validation dataset.

   Discuss your findings using suitable visualizations. (2 points)

d) Repeat the previous subtask, this time initializing the word embedding layer with the Glove embeddings (code provided; that's *pretraining with finetuning*). **Discuss** your findings using suitable visualizations. (2 points)

e) Repeat the previous subtask, this time additionally freezing the word embeddings to the Glove embeddings (code provided; that's *pretraining without finetuning*). **Discuss** your findings using suitable visualizations. (2 points)

f) **Optional.** Train the model as best as you can. You may use fine-tuning, different HPs, learning rate scheduling provided by Lightning, a different optimizer etc. **Report** your findings.

[3] This file contains a subset of the complete GloVe embeddings available at `http://nlp.stanford.edu/data/glove.6B.zip`